

scripts/11_code_to_pdf.py

220 lines

```
1 """
2 11_code_to_pdf.py
3 =====
4 Renders every Python source file into a readable, syntax-highlighted PDF.
5
6 Exists for one reason: the hackathon portal only accepts PDF or zip, and its own
7 instructions say "in case of errors uploading zip files, convert/print all files
8 to pdf and upload." This script is that fallback, generated automatically so the
9 PDFs can never drift from the actual code.
10
11 Produces:
12 submission/code_pdfs/<flat-name>.py.pdf one PDF per source file
13 submission/ALL_SOURCE_CODE.pdf every file concatenated, in one PDF,
14 with a table of contents - upload
15 this single file if a zip won't go through
16 """
17
18 from __future__ import annotations
19
20 import sys
21 from pathlib import Path
22
23 ROOT = Path(__file__).resolve().parents[1]
24 sys.path.insert(0, str(ROOT / "src"))
25
26 from pygments import highlight
27 from pygments.lexers import PythonLexer
28 from pygments.formatters import HtmlFormatter
29 from reportlab.lib import colors
30 from reportlab.lib.pagesizes import LETTER
31 from reportlab.lib.styles import ParagraphStyle
32 from reportlab.lib.units import inch
33 from reportlab.platypus import (
34 SimpleDocTemplate, Paragraph, Spacer, HRFflowable, PageBreak,
35 Table, TableStyle,
36 )
37 from reportlab.lib.enums import TA_LEFT
38 import re
39 from html.parser import HTMLParser
40
41 OUT_DIR = ROOT / "submission" / "code_pdfs"
42 COMBINED_PDF = ROOT / "submission" / "ALL_SOURCE_CODE.pdf"
43
44 # Files in a deliberate reading order: interface/config first, then the physics,
45 # then model + controller, then the scripts pipeline in run order, then tests.
46 FILE_ORDER = [
47 "src/simulator_interface.py",
48 "src/config.py",
49 "src/well_simulator.py",
50 "src/model.py",
51 "src/controller.py",
52 "src/plotting.py",
53 "scripts/01_reference_dataset_eda.py",
54 "scripts/02_step_tests.py",
55 "scripts/03_identify_model.py",
56
57 "scripts/04_run_scenarios.py",
58 "scripts/05_export_dashboard_data.py",
59 "scripts/06_robustness_study.py",
60 "scripts/07_capture_dashboard.py",
61 "scripts/08_export_report_pdf.py",
62 "scripts/09_build_presentation.py",
63 "scripts/10_build_ppt_assets.py",
64 "scripts/11_code_to_pdf.py",
65 "tests/test_all.py",
66 ]
67
68 INK = colors.HexColor("#0f172a")
```

```

68 DIM = colors.HexColor("#64748b")
69 BLUE = colors.HexColor("#1d4fd8")
70 LINE_BG = colors.HexColor("#f6f8fa")
71
72 # Pygments token -> reportlab colour, tuned for a light background (print-friendly)
73 TOKEN_COLORS = {
74     "Token.Keyword": colors.HexColor("#cf222e"),
75     "Token.Keyword.Namespace": colors.HexColor("#cf222e"),
76     "Token.Keyword.Constant": colors.HexColor("#0550ae"),
77     "Token.Name.Builtin": colors.HexColor("#8250df"),
78     "Token.Name.Builtin.Pseudo": colors.HexColor("#8250df"),
79     "Token.Name.Function": colors.HexColor("#8250df"),
80     "Token.Name.Class": colors.HexColor("#8250df"),
81     "Token.Name.Decorator": colors.HexColor("#8250df"),
82     "Token.String": colors.HexColor("#0a3069"),
83     "Token.String.Doc": colors.HexColor("#6e7781"),
84     "Token.Number": colors.HexColor("#0550ae"),
85     "Token.Comment": colors.HexColor("#6e7781"),
86     "Token.Operator": colors.HexColor("#cf222e"),
87     "Token.Punctuation": INK,
88     "Token.Name": INK,
89 }
90
91
92 def escape(s: str) -> str:
93     return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
94
95
96 def tokenize_to_markup(code: str) -> list[str]:
97     """Convert Python source into reportlab mini-XML markup, one string per line,
98     preserving Pygments' syntax colouring."""
99     from pygments.lexers import PythonLexer as Lexer
100     from pygments import lex
101
102     lines_markup = [""]
103     for ttype, value in lex(code, Lexer()):
104         colour = None
105         t = str(ttype)
106         while t and colour is None:
107             colour = TOKEN_COLORS.get(t)
108             t = t.rsplit(".", 1)[0] if "." in t else ""
109         colour = colour or INK
110         hexcol = colour.hexval()[2:] if hasattr(colour, "hexval") else "0f172a"
111
112         parts = value.split("\n")
113         for i, part in enumerate(parts):
114             if part:
115                 lines_markup[-1] += f'<font color="{hexcol}">{escape(part)}</font>'
116             if i < len(parts) - 1:
117                 lines_markup.append("")
118         return lines_markup
119
120
121 def build_file_flowables(rel_path: str, code_style, header_style) -> list:
122     path = ROOT / rel_path
123     text = path.read_text()
124     n_lines = text.count("\n") + 1
125
126     flow = []
127     flow.append(Paragraph(escape(rel_path), header_style))
128     flow.append(Paragraph(f"{n_lines} lines", ParagraphStyle(
129         "meta", fontName="Helvetica", fontSize=8.5, textColor=DIM, spaceAfter=8)))
130     flow.append(HRFlowable(width="100%", thickness=0.7, color=colors.HexColor("#d0d7de"), spaceAfter=8))
131
132     lines = tokenize_to_markup(text)
133     # Group ~55 lines per Paragraph so long files don't choke reportlab's layout engine
134     chunk = []
135     for i, line in enumerate(lines, start=1):
136         num = f'<font color="#8c959f">{i}>4 </font>'
137         chunk.append(num + (line or "&nbsp;"))

```

```

138 if len(chunk) >= 55:
139     flow.append(Paragraph("<br/>".join(chunk), code_style))
140     chunk = []
141 if chunk:
142     flow.append(Paragraph("<br/>".join(chunk), code_style))
143 return flow
144
145
146 def main():
147     OUT_DIR.mkdir(parents=True, exist_ok=True)
148
149     header_style = ParagraphStyle(
150         "fheader", fontName="Helvetica-Bold", fontSize=12.5, textColor=BLUE,
151         spaceAfter=2, alignment=TA_LEFT,
152     )
153     code_style = ParagraphStyle(
154         "code", fontName="Courier", fontSize=7.6, leading=9.6,
155         textColor=INK, backColor=LINE_BG, borderPadding=6, spaceAfter=6,
156     )
157     toc_title_style = ParagraphStyle("toctitle", fontName="Helvetica-Bold", fontSize=18,
158         textColor=INK, spaceAfter=4)
159     toc_sub_style = ParagraphStyle("tocsub", fontName="Helvetica", fontSize=10,
160         textColor=DIM, spaceAfter=18)
161     toc_row_style = ParagraphStyle("tocrow", fontName="Helvetica", fontSize=10.5,
162         textColor=INK)
163
164     files = [f for f in FILE_ORDER if (ROOT / f).exists()]
165     missing = [f for f in FILE_ORDER if not (ROOT / f).exists()]
166
167     if missing:
168         print(f" NOTE: not found, skipped: {missing}")
169
170     # ---- individual per-file PDFs -----
171     print(f"Rendering {len(files)} individual PDFs to {OUT_DIR.relative_to(ROOT)}/ ...")
172     for rel_path in files:
173         flat_name = rel_path.replace("/", "_") + ".pdf"
174         doc = SimpleDocTemplate(
175             str(OUT_DIR / flat_name), pagesize=LETTER,
176             topMargin=0.6 * inch, bottomMargin=0.6 * inch,
177             leftMargin=0.55 * inch, rightMargin=0.55 * inch,
178             title=rel_path,
179         )
180         doc.build(build_file_flowables(rel_path, code_style, header_style))
181         print(f" {flat_name}")
182
183     # ---- one combined PDF with a table of contents -----
184     print(f"\nRendering combined PDF: {COMBINED_PDF.relative_to(ROOT)}")
185     story = []
186     story.append(Paragraph("Autonomous Production Choke Controller", toc_title_style))
187     story.append(Paragraph(
188         "Full source code &mdash; every Python file in the project, in one PDF. "
189         f"{len(files)} files, {sum((ROOT / f).read_text().count(chr(10)) + 1 for f in files)} lines total.",
190         toc_sub_style))
191
192     toc_rows = [[Paragraph(f"{i+1}.", toc_row_style), Paragraph(escape(f), toc_row_style)]
193         for i, f in enumerate(files)]
194     toc_table = Table(toc_rows, colWidths=[0.45 * inch, 6.0 * inch])
195     toc_table.setStyle(TableStyle([
196         ("VALIGN", (0, 0), (-1, -1), "TOP"),
197         ("TOPPADDING", (0, 0), (-1, -1), 2),
198         ("BOTTOMPADDING", (0, 0), (-1, -1), 2),
199     ]))
200     story.append(toc_table)
201     story.append(PageBreak())
202
203     for rel_path in files:
204         story.extend(build_file_flowables(rel_path, code_style, header_style))
205         story.append(PageBreak())
206     if story and isinstance(story[-1], PageBreak):
207         story.pop()

```

```
208 doc = SimpleDocTemplate(
209     str(COMBINED_PDF), pagesize=LETTER,
210     topMargin=0.6 * inch, bottomMargin=0.6 * inch,
211     leftMargin=0.55 * inch, rightMargin=0.55 * inch,
212     title="Autonomous Production Choke Controller - Source Code",
213 )
214 doc.build(story)
215 print(f" wrote {COMBINED_PDF} ({COMBINED_PDF.stat().st_size / 1024:.0f} KB)")
216
217
218 if __name__ == "__main__":
219     main()
220
```